

Conception d'Interface Web

JavaScript (1/2)

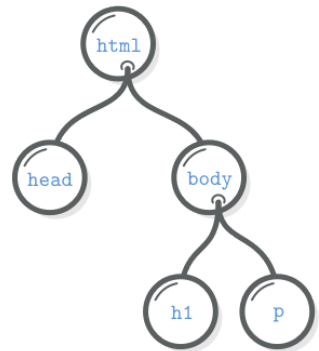
Simon BERNARD

simon.bernard@univ-rouen.fr



Présentation de JavaScript

1. Structuration/organisation du contenu (texte, images, vidéos) : HTML
2. Présentation (visuelle) du contenu : CSS
3. **Rendre ce contenu interactif : JavaScript**



HTML



CSS



JAVASCRIPT

Présentation de JavaScript

- Langage de programmation interprété
- S'exécute dans un environnement équipé d'un moteur JavaScript (e.g. navigateur)
- Principalement utilisé côté client (mais il existe des environnements côté serveur, comme Node.js)
- Le seul langage de programmation activé par défaut dans les navigateurs
- Permet de:
 1. Manipuler le contenu des pages web programmatiquement
 2. Réagir aux événements utilisateur (clics, saisie de texte, etc.)
 3. Communiquer avec des serveurs distants (AJAX)

Bref historique

- 1995: création par Brendan Eich pour Netscape Navigator
- 1997: standardisation sous le nom ECMAScript (ES)
- 1999: IE propose son propre langage JScript (début du déclin de JS)
- **2008: Compilateur JIT (chrome). Les moteurs deviennent beaucoup plus performants**
- 2009: ES5, retour de JS au premier plan
- 2015: ES6, évolution majeure, ajout de nombreuses fonctionnalités modernes
- Depuis 2016: nouvelles versions annuelles (ES7, ES8, ...)

Aujourd'hui (2025), 98,9% des sites web utilisent JavaScript ([source](#))

Compilation *Just-In-Time*

- Initialement, langage interprété, mais aujourd'hui les moteurs utilisent la compilation *Just-In-Time* (JIT)
 1. Le moteur analyse le code du script
 2. Le compile en code machine natif
 3. Exécute ce code aussi rapidement que possible
- Permet de nombreuses optimisations à l'exécution
- Exemples de moteurs JavaScript:
 - V8 (Chromium et donc Chrome, Edge, Opera, Brave)
 - SpiderMonkey (Firefox)
 - JavaScriptCore (Safari)
 - Chakra (IE et anciennement utilisé par Edge)

Plus d'infos: https://fr.wikipedia.org/wiki/Moteur_JavaScript

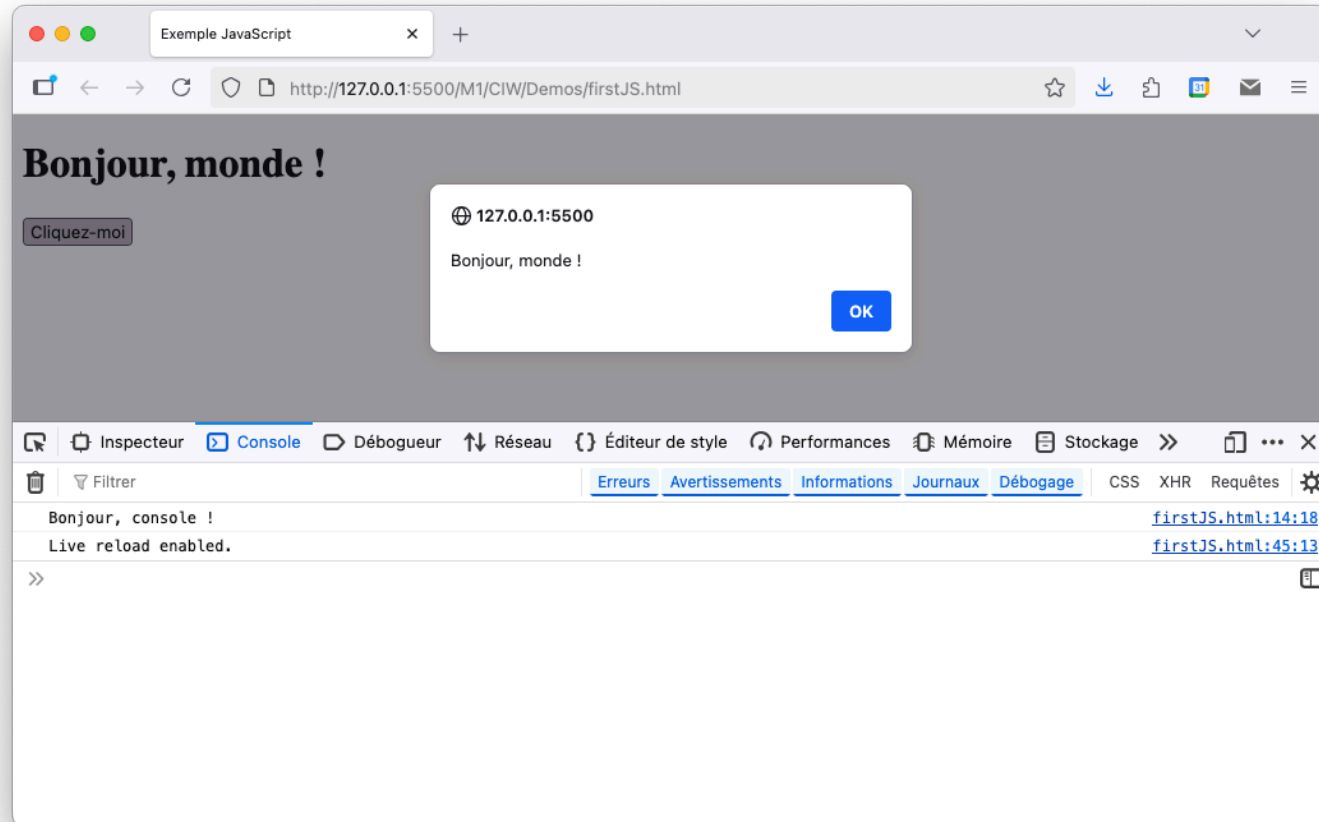
Fonctionnement de JavaScript

1. Intégrer du code JS dans une page HTML

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <title>Exemple JavaScript</title>
5   <script>
6     function direBonjour() {
7       alert("Bonjour, monde !");
8     }
9   </script>
10 </head>
11 <body>
12   <h1>Bonjour, monde !</h1>
13   <script>
14     console.log("Bonjour, console !");
15   </script>
16   <button onclick="direBonjour()">Cliquez-moi</button>
17   <script src="mon_script.js"></script>
18 </body>
19 </html>
```

Fonctionnement de JavaScript

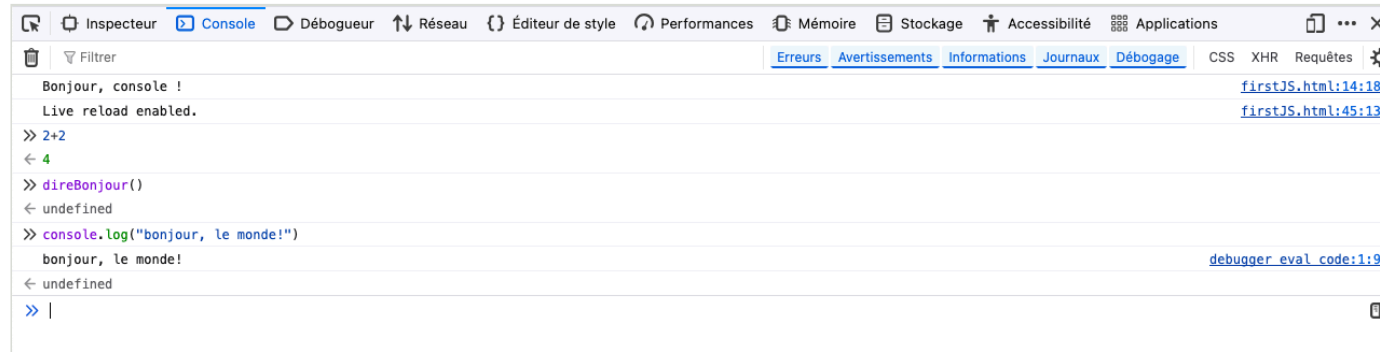
2. Chargement et exécution de la page dans un navigateur



Fonctionnement de JavaScript

3. Si besoin, déboguer via les outils du navigateur:

➤ Console interactive



➤ Débogueur



Intégration de JS dans une page HTML

- Balise `<script>` dans le `<head>` ou le `<body>`
- Contient du code JavaScript ou `src` pour un fichier externe
- Exemples:

```
<script>
  let nom = prompt("Quel est votre nom ?");
  let show = confirm("Voulez-vous afficher votre nom ?");
  if (show) {
    alert("Bonjour " + nom);
    document.write("<p>Bonjour " + nom + "</p>");
  } else {
    alert("D'accord, au revoir");
  }
  console.log("Le nom saisi est: " + nom);
</script>
<script src="mon_script.js"></script>
```

- Les scripts sont chargés et exécutés dans l'ordre d'apparition

Intégration de JS dans une page HTML

- HTML autorise aussi les scripts *inline*

```
<head>
  <script>
    function dialog() {
      alert("JavaScript dans une fonction");
    }
  </script>
</head>
<body>
  <a href="Javascript:dialog()">Message d'alerte</a>
  <button onclick="dialog()">Say hello!</button>
</body>
```

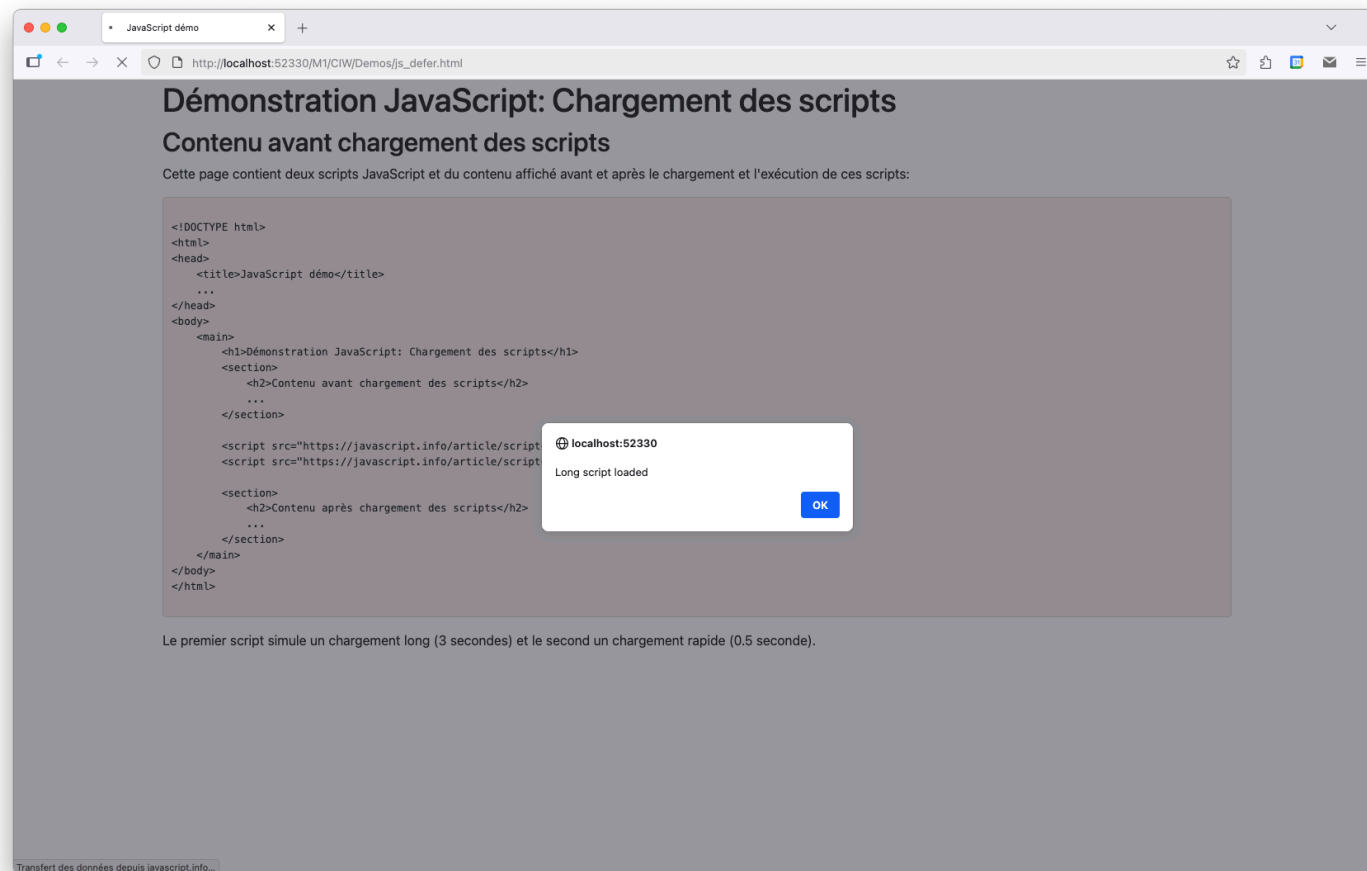
- Permet de déclencher du JS en réaction à des événements utilisateur
- Plusieurs types d'événements possibles (**onload**, **onclick**, **onchange**, etc.)
- **Non recommandé!** Nous verrons une meilleure méthode plus tard

Chargement des scripts

- Chargement d'une page HTML par le navigateur:
 1. Réception et analyse (parsing)
 2. Construction du DOM (Document Object Model)
 3. Construction du CSSOM (CSS Object Model)
 4. Rendu et affichage de la page
- Étape 2 exécutée au fil de la lecture du document HTML
- Si lien vers une ressource externe (CSS, JS, images) la construction du DOM est interrompue
- Le téléchargement, le chargement et l'exécution des scripts JS sont bloquants
- Peut considérablement impacter les temps de chargements et l'expérience utilisateur

Chargement des scripts

Exemple

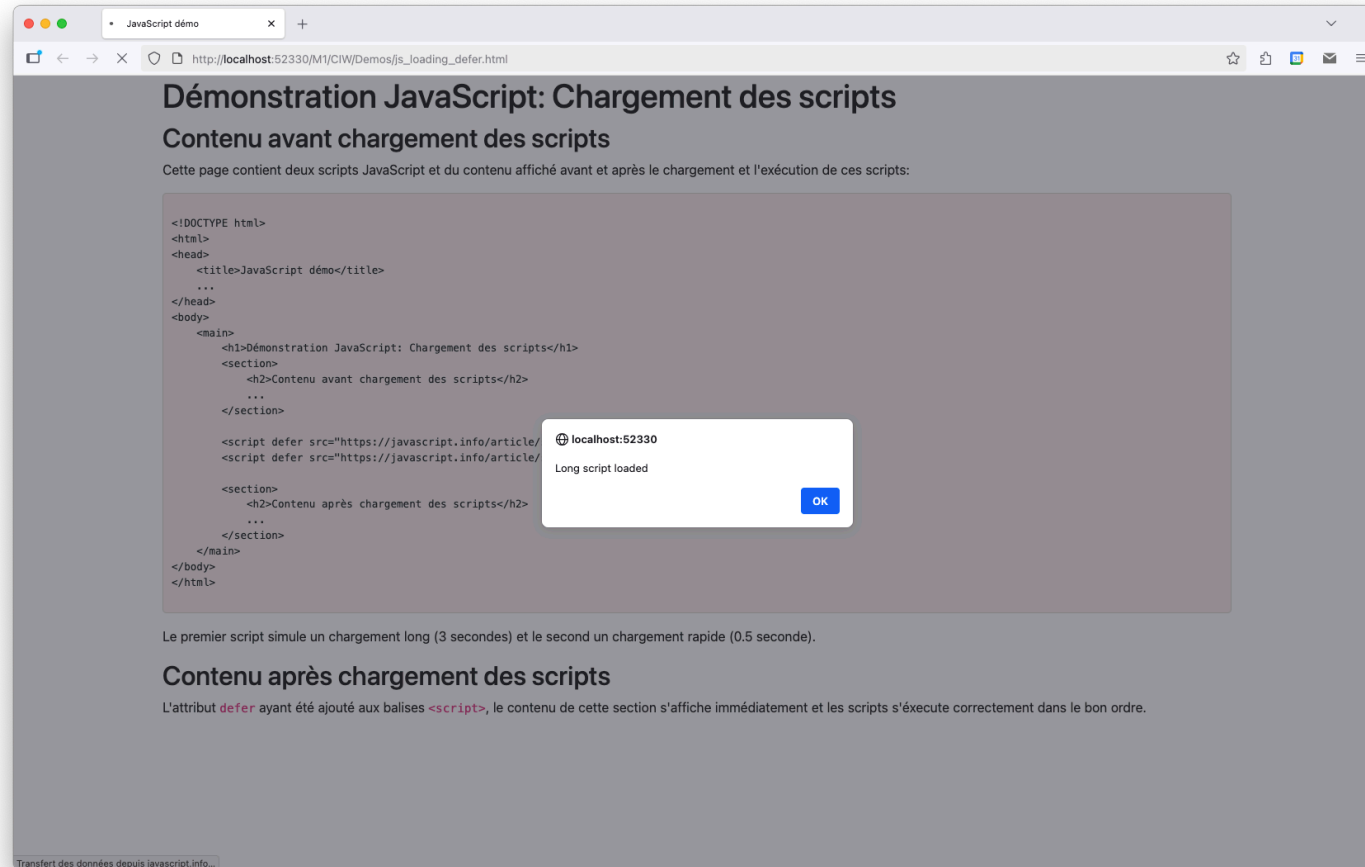


Chargement des scripts

- Solutions pour éviter le blocage:
 1. Placer les `<script>` en fin de `<body>`
 2. Attribut `defer`: script chargé en parallèle et exécuté après la construction du DOM
 3. Attribut `async`: script chargé en parallèle et exécuté dès qu'il est prêt
- avec `defer`, l'ordre des scripts est préservé et le contenu s'affiche avant l'exécution des scripts
- avec `async`, l'ordre des scripts n'est pas préservé et les scripts sont exécutés indépendamment du rendu

Chargement des scripts

Exemple



Le langage JavaScript

Variables

- JS est faiblement typé: les variables n'ont pas de type fixe
- Déclaration

```
let nom;  
nom = "John Doe";  
let ville = "Rouen";  
  
const PI = 3.14159;  
PI = 42; // Erreur
```

- Typage dynamique: le type est déterminé à l'affectation

```
let variable = 42;  
variable = "Bonjour";  
variable = true;
```

Variables

- Avant 2015:

```
var age = 30;  
age = 31;
```

- Autorise l'utilisation avant la déclaration (*hoisting*)

```
console.log(age); // affiche "undefined" (pas d'erreur)  
var age = 30;
```

- Autorise l'utilisation en dehors de blocs

```
for (var i = 0; i < 11; i++) {  
    var sum = i * 3.14;  
}  
console.log(i); // affiche 11  
console.log(sum); // affiche 31.4
```

- Toujours pris en charge mais à éviter!

Types primitifs

- **Boolean**: valeurs **true** ou **false**

```
let ok = true;  
let isGreater = (5 > 3); // true
```

- **Number**: valeurs numériques (entier ou décimaux) compris entre **Number.MIN_VALUE** et **Number.MAX_VALUE** + 3 valeurs particulières (**NaN**, **Infinity**, **-Infinity**)

```
let notANumber = 0 / 0; // NaN  
let inf = 1 / 0; // Infinity
```

- **String**: chaîne de caractères (délimitées par des guillemets simples, doubles ou backticks)

```
let nom = "John Doe";  
let message = `Bonjour, ${nom}!`;
```

- **Undefined**: valeur unique **undefined**, qui est la valeur par défaut des variables non initialisées

Objets

- JS est un langage orienté objet (orienté prototype)
- Type **Object**: collection de paires clé/valeur

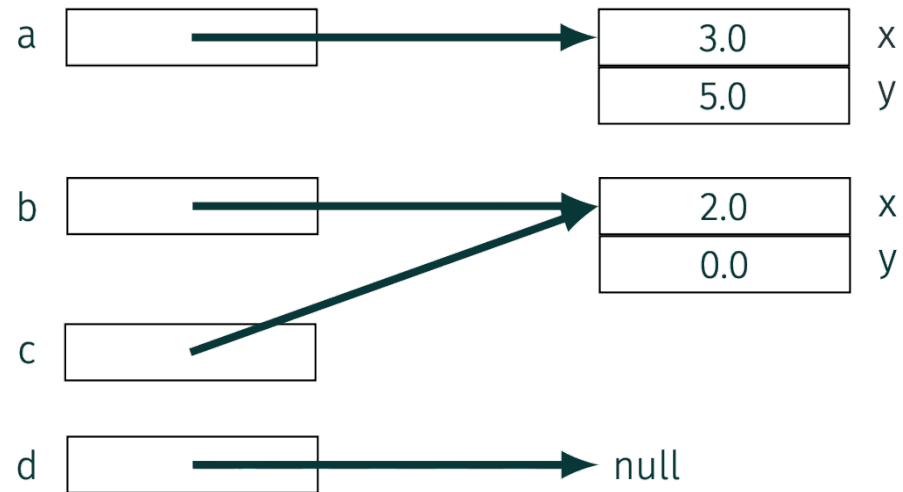
```
let personne = {  
  nom: "John Doe",  
  age: 30,  
  ville: "Rouen"  
};  
console.log(personne.nom); // "John Doe"  
console.log(personne["age"]); // 30  
  
let empty = {}; // objet vide  
let empty2 = new Object(); // équivalent
```

- valeur **null**: absence intentionnelle de valeur

Objets

- Les objets sont manipulés par référence (comme en Java)

```
let a = { x:3.0, y:5.0 };  
let b = { x:2.0, y:0.0 };  
let c = b;  
let d = null;
```



Objets prédéfinis

- JS prédéfinit des "sous-types" d'**Object**

Type	Description	Exemples
Array	Tableau (liste ordonnée d'éléments)	[1, 2, 3], ["a", "b", "c"]
Date	Date et heure	<code>new Date()</code> , <code>new Date("2023-10-01")</code>
RegExp	Expressions régulières	<code>/abc/</code> , <code>new RegExp("abc")</code>
Error	Objets d'erreur (exceptions)	<code>new Error("message d'erreur")</code>
Math	Objets mathématiques (fonctions et constantes)	<code>Math.PI</code> , <code>Math.sqrt(4)</code>
Function	Fonctions	<code>function() {}</code> , <code>const f = () => {}</code>

Liste complète ici: [MDN Web Docs](https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects)

Objets prédéfinis

Tableaux

- Peut contenir n'importe quels types d'éléments

```
let nombres = [1, 2, 3, 4, 5];  
let lettres = ["a", "b", "c"];  
let mixte = [1, "deux", true, {x:3}, [1,2,3]];
```

- Accès aux éléments par indice (base 0)

```
console.log(nombres[0]); // 1  
nombres[1] = 42;  
console.log(nombres.length); // 5
```

- Nombre d'éléments dynamique

```
nombres.push(6); // ajoute 6 à la fin  
nombres.pop(); // enlève le dernier élément  
nombres.shift(); // enlève le premier élément  
nombres.unshift(0); // ajoute 0 au début
```

- Nombreuses méthodes prédéfinies (voir [documentation](#))

Opérateurs et instructions de contrôle

- Les mêmes que C/Java
 - Arithmétiques: `+`, `-`, `*`, `/`, `%`
 - Comparaison: `==`, `!=`, `<`, `>`, `<=`, `>=`
 - Logiques: `&&`, `||`, `!`
 - Conditionnelle: `if`, `else`, `switch`
 - Boucles: `for`, `while`, `do...while`
 - Opérateur ternaire: `condition ? expr1 : expr2`
- + quelques uns spécifiques à JS
 - Exposant: `**`
 - Comparaison stricte: `===`, `!==` (vérifie aussi le type)
 - Opérateur *nullish coalescing*: `a ?? b` (retourne `a` si non null/undefined, sinon `b`)

Conversion implicites

- + devient l'opérateur de concaténation si un des opérandes est un **string**

```
let s = "my" + "string"
console.log(s); // "mystring"
console.log("1" + 2); // "12"
console.log(1 + "2"); // "12"
console.log(2 + 2 + "1"); // "41"
console.log("50" + null); // "50null"
```

- les autres opérateurs convertissent les chaînes en nombres

```
console.log("2" - 1); // 1
console.log("2" + 1); // "21"
```

- D'où le besoin d'un opérateur d'égalité stricte

```
console.log(3 == "3"); // true
console.log(3 === "3"); // false
```

Toutes les conversions implicites: https://www.w3schools.com/js/js_type_conversion.asp

Les fonctions en JavaScript

Fonctions

➤ Déclaration:

```
function name(par1[, par2[, ..., parN]]) {  
    /* corps de la fonction */  
    ...  
    [return valeur;]  
}
```

- Ce qui est entre `[]` est optionnel
- Pas de types de retour ni de types de paramètres
- Les arguments sont passés par valeur, sauf pour les objets
- Si pas de `return`, la fonction retourne `undefined`

Fonctions

- Invocation: les arguments sont facultatifs. En cas d'absence, les paramètres sont **undefined**

```
function optional(par1, par2) {  
    console.log("par1=" + par1); // affiche la valeur de par1 dans la console  
    console.log("par2=" + par2); // affiche la valeur de par2 dans la console  
}  
optional(); // affichage "par1=undefined" puis "par2=undefined"  
optional("Hello"); // affichage "par1=Hello" puis "par2=undefined"
```

- Possibilité de spécifier une valeur par défaut

```
function withDefault(par1, par2 = 42) {  
    console.log("par1=" + par1);  
    console.log("par2=" + par2);  
}  
withDefault("Hello"); // affichage "par1=Hello" puis "par2=42"  
withDefault("Hello", 7); // affichage "par1=Hello" puis "par2=7"
```

Fonctions

- Dans une page HTML, les déclarations doivent précéder leur utilisation

```
<!DOCTYPE html>
<html lang="fr">
<head>
  ...
  <script src="fonction1.js"></script>
</head>
<body>
  <script>
    console.log(fonction1()); // ok
    console.log(fonction2()); // erreur
  </script>
  <script src="fonction2.js"></script>
</body>
</html>
```

Fonctions

- Dans un fichier `.js`, elles peuvent être déclarées dans n'importe quel ordre

```
function cube(x) {  
  return x * square(x);  
}  
function square(x) {  
  return x * x;  
}  
console.log(cube(3)); // 27
```

- Dû au fait que les déclarations sont traitées au chargement, avant l'exécution.

Fonctions expressions

- En JS, les fonctions sont des types d'objets
- On peut affecter une fonction à une variable

```
function sayHello() {  
    console.log( "Hello" );  
}  
// autre syntaxe équivalente:  
let sayHello = function() {  
    console.log( "Hello" );  
};  
sayHello(); // invocation identique pour les deux syntaxes
```

- La deuxième syntaxe est appelée *fonction expression*
- Attention au ; à la fin

Fonctions expressions

- On peut affecter une fonction à plusieurs variables (référence)

```
let sayHelloAgain = sayHello;  
sayHelloAgain(); // même résultat que sayHello()
```

- On peut également passer une fonction en argument d'une autre fonction

```
function callFunction(f) {  
    f();  
}  
callFunction(sayHello); // affiche "Hello"
```

ou

```
callFunction(  
    function() {  
        console.log("Hello from anonymous function");  
    }  
);
```

Callback

- Une fonction passée en argument d'une autre fonction est appelée un *callback*

```
function ask(question, yes, no) {  
  if (confirm(question)) yes();  
  else no();  
}  
  
function showOk() {  
  alert( "You agreed." );  
}  
function showCancel() {  
  alert( "You canceled the execution." );  
}  
ask("Do you agree?", showOk, showCancel);
```

OU

```
ask("Do you agree?",  
  function() { alert("You agreed."); },  
  function() { alert("You canceled the execution."); }  
);
```

Fonction fléchée

- Syntaxe plus concise pour les fonctions expressions

```
let sum = (a, b) => a + b;  
console.log( sum(1, 2) ); // 3
```

- Si un seul paramètre, les parenthèses sont optionnelles

```
let square = x => x * x;  
console.log( square(3) ); // 9
```

- Si pas de paramètre, parenthèses obligatoires

```
let sayHello = () => console.log("Hello");  
sayHello(); // Hello
```

- Si plusieurs instructions, utiliser des accolades et le mot-clé **return** si besoin

```
let sumAndLog = (a, b) => {  
  let result = a + b;  
  console.log(result);  
  return result;  
};  
sumAndLog(2, 3); // affiche 5 et retourne 5
```

Les bonnes pratiques

Deux JavaScript: "vieux" vs "moderne"

- Avant ES5:
 - très permissif et mécanismes exotiques (e.g. *hoisting*)
 - prise en charge inégale des navigateurs
 - code souvent verbeux et difficile à maintenir
- ES5:
 - évolution majeures de fonctionnalités existantes
 - mais introduit des problèmes de rétrocompatibilité (mode **strict**)
- Depuis ES6 (2015) et versions ultérieures:
 - ajout de nombreuses fonctionnalités (e.g. classes, modules, promesses, async/await, etc.)
 - amélioration de la lisibilité et de la maintenabilité du code
 - adoption généralisée mais non prises en charges par les "vieux" navigateurs

Deux JavaScript: "vieux" vs "moderne"

- Par défaut, les navigateurs désactivent les fonctionnalités modernes non compatibles avec le "vieux" JS
- Pour activer le JS "moderne": directive `use strict` à inclure au début d'un script

```
"use strict";  
  
// code JS ci-dessous peut utiliser les fonctionnalités modernes  
let x = 3.14;  
...
```

- Plusieurs fonctionnalités modernes activent automatiquement ce mode, e.g. les classes et les modules
- `use strict` est souvent omise car implicitement activée par des scripts modernes

Transpiler et polyfills

➤ Problèmes de retrocompatibilité prépondérants en JS

➤ Solutions:

1. *Transpiler*: outil qui convertit du code JS moderne en code compatible avec les anciens navigateurs (e.g. Babel)

```
// avant d'exécuter le transpileur:  
height = height ?? 100; // opérateur introduit en 2020  
// après avoir exécuté le transpileur:  
height = (height !== undefined && height !== null) ? height : 100;
```

2. *Polyfills*: bibliothèques qui ajoutent des fonctionnalités modernes aux anciens navigateurs

```
if (!Math.trunc) { // si une telle fonction n'existe pas  
  Math.trunc = function(number) {  
    return number < 0 ? Math.ceil(number) : Math.floor(number);  
  };  
}
```

Permissivité

- JS est un langage très permissif et piegeux: [wtfjs](#) ou



- Un code JS peut vite devenir illisible, incompréhensible et peu performant
- Solutions: utilisez des guides de styles, des *linters*, des IDE dédiés, un débogueur

Liens utiles

- [JavaScript.info](https://javascript.info): Un site très complet et abordable pour apprendre JavaScript
- [Web.dev](https://web.dev/): Séries d'articles pour approfondir JavaScript
- [MDN Web Docs](https://developer.mozilla.org/fr/docs/Web/JavaScript): Références et tutoriels sur JavaScript
- [freeCodeCamp](https://www.freecodecamp.org/): Exercices interactifs
- [CodeWars](https://www.codewars.com/): Exercices de code
- [The Net Ninja](https://www.youtube.com/channel/UCbXg17UCgJpUc4qRUhE9l9Q): Vidéos pédagogiques
- [ESLint](https://eslint.org/): un des *linters* les plus populaires